

CS143 Spring 2024 – Written Assignment 1

Due Thursday, April 18, 2024 11:59 PM PDT

This assignment covers regular languages, finite automata, and lexical analysis. You may discuss this assignment with other students and work on the problems together. However, your write-up should be your own individual work, and you should indicate in your submission who you worked with, if applicable. Assignments can be submitted electronically through Gradescope as a PDF by 11:59 PM PDT on the due date. Please review the course policies for more information: <http://web.stanford.edu/class/cs143/policies/index.html>. A $\text{\LaTeX}$ template for writing your solutions is available on the course website. To create finite automata diagrams, you can either use the *TikZ* package directly by following the examples in the template, or a tool like <https://madebyevan.com/fsm/>.

1. Write regular expressions for the following languages over the alphabet $\Sigma = \{0, 1\}$.

(a) The set of strings beginning with 0 and having an odd numbered length.

Solution: $0((0|1)(0|1))^*$

(b) The set of all strings that contain less than three 1's.

Solution: $0^*1?0^*1?0^*$

(c) The set of strings where 0's and 1's appear only in alternating groups of odd numbered length. Examples of strings in the language: ε , 0, 0111, 0001000, and 101000. Examples of strings not in the language: 11000, 111011, and 00.

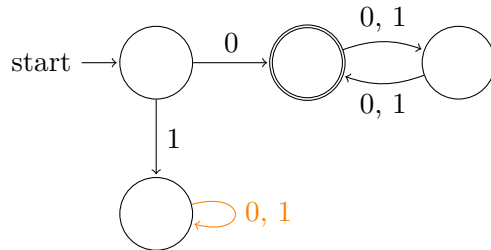
Solution: $((1(11)^*)|(0(00)^*))^*(0(00)^*)?(1(11)^*0(00)^*)(1(11)^*)?$

2. Draw DFAs for each of the languages from question 1. Note that a DFA must have a transition defined for every state and symbol pair. You must take this fact into account for your transformations. Your DFAs should not have more than 10 states. Submissions with unnecessarily complex DFAs may not receive full credit.

Notice that a short regular expression does not automatically imply a DFA with few states, nor vice versa.

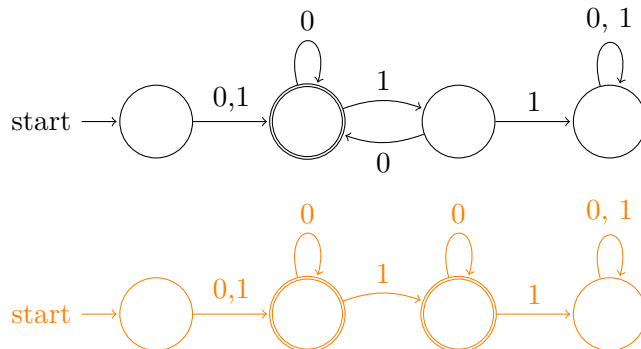
- (a) The set of strings beginning with 0 and having an odd numbered length.

Solution: $0((0|1)(0|1))^*$



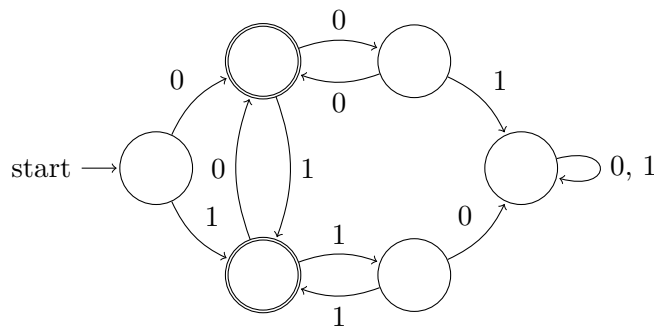
- (b) The set of all strings that contain less than three 1's.

Solution: $0^*1?0^*1?0^*$



- (c) The set of strings where 0's and 1's appear in alternating groups of odd numbered length.

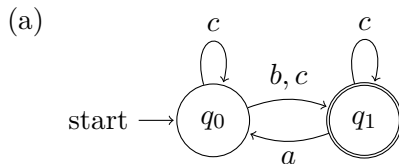
Solution: $(0(00)^*)?(1(11)^*0(00)^*)(1(11)^*)?$



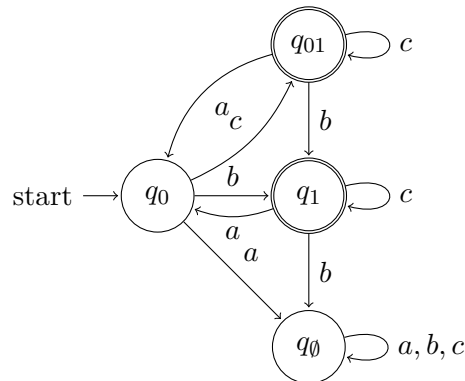
3. Using the techniques covered in class, transform the following NFAs over the alphabet $\{a, b, c\}$ into DFAs. Your DFAs should not have more than 10 states. Note that a DFA must have a transition defined for every state and symbol pair, whereas a NFA need not. You must take this fact into account for your transformations. Hint: Is there a subset of states the NFA transitions to when fed a symbol for which the set of current states has no explicit transition?

Also include a mapping from each state of your DFA to the corresponding states of the original NFA. Specifically, a state q of your DFA maps to the set of states Q of the NFA such that an input string stops at q in the DFA if and only if it stops at one of the states in Q in the NFA.

Tip: for readability, states in the DFA may be labeled according to the set of states they represent in the NFA. For example, state q_{012} in the DFA would correspond to the set of states $\{q_0, q_1, q_2\}$ in the NFA, whereas state q_{13} would correspond to set of states $\{q_1, q_3\}$ in the NFA.

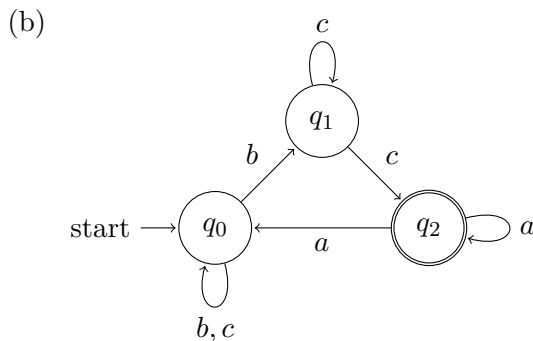


Solution:

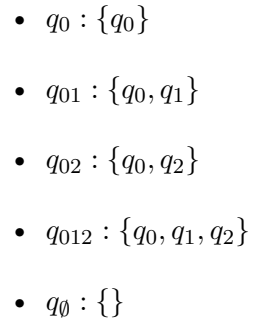


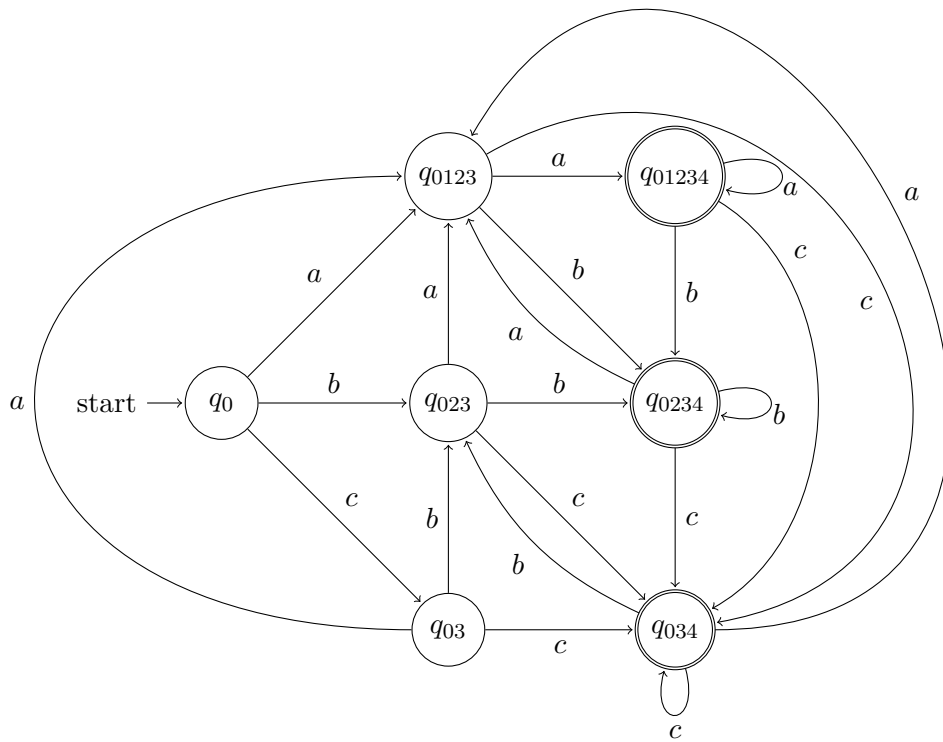
Corresponding states
(DFA to NFA)

- $q_0 : \{q_0\}$
- $q_1 : \{q_1\}$
- $q_{01} : \{q_0, q_1\}$
- $q_\emptyset : \{\}$



Solution:





Corresponding states (DFA to NFA)

- $q_0 : \{q_0\}$
- $q_{03} : \{q_0, q_3\}$
- $q_{023} : \{q_0, q_2, q_3\}$
- $q_{034} : \{q_0, q_3, q_4\}$
- $q_{0123} : \{q_0, q_1, q_2, q_3\}$
- $q_{0234} : \{q_0, q_2, q_3, q_4\}$
- $q_{01234} : \{q_0, q_1, q_2, q_3, q_4\}$

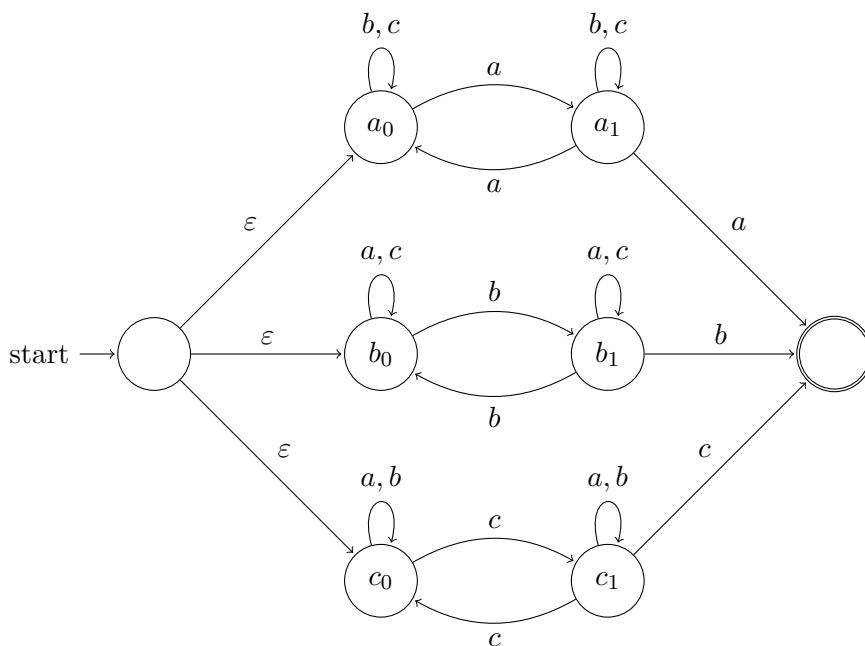
4. Let L be a language over $\Sigma = \{a, b, c\}$, such that string w is in L if and only if $w \neq \varepsilon$ and the last character in w appears an even number of times in w

Examples of strings in L : $aa, abacaba, bab$.

Examples of strings **not** in L : $\varepsilon, a, abcabca, bbb$.

Draw an NFA for L . Your solution should have no more than 15 states.

Solution:



从最终状态向前逆推会比较简单

5. Consider the following tokens and their associated regular expressions, given as a **flex** scanner specification:

```
%%  
(01|10)                printf("apple");  
(10)?0?                printf("banana");  
(0110+|1001*1)        printf("coconut");
```

Give an input to this scanner such that the output string is $((\text{apple})^3(\text{banana})^4)^2((\text{apple})^3\text{coconut})^2$, where A^i denotes A repeated i times. (And, of course, the parentheses are not part of the output.) You may use similar shorthand notation in your answer.

Solution: $((01)^3(0)^4)^2(01)^30110(10)^31001$

flex 同时有多个匹配的话, 选择最长的匹配项。如果有多个长度相同的匹配项, 则选择靠前的规则来进行匹配。因此, 表示 apple 的 01 和 10 不能交替出现。因为 0110 和 0110 均会匹配到 coconut。同理, 01 后不能直接跟 1001, 10 后不能直接跟 0110 来表示 (apple)(coconut), 1001 后也不能直接跟 10, 0110 后也不能跟 01 来表示 (coconut)(apple)。

同时 10 表示不了 banana, 因为字符长度相同时, flex 会优先选择靠前的规则。

如果用 100 来表示 banana, 则不能连续表示, 因为 (1001)00 会匹配为 coconut

6. Recall from the lecture that, when using regular expressions to scan an input, we resolve conflicts by taking the largest possible match at any point. That is, if we have the following **flex** scanner specification:

```
%%  
do { return T_Do; }  
[A-Za-z_][A-Za-z0-9_]* { return T_Identifier; }
```

and we see the input string “dot”, we will match the second rule and emit T_Identifier for the whole string, not T_Do.

However, it is possible to have a set of regular expressions for which we can tokenize a particular string, but for which taking the largest possible match will fail to break the input into tokens. Give an example of no more than two regular expressions and an input string such that: a) the string can be broken into substrings, where each substring matches one of the regular expressions, b) our usual lexer algorithm, taking the largest match at every step, will fail to break the string in a way in which each piece matches one of the regular expressions. Explain how the string can be tokenized and why taking the largest match won't work in this case.

As a challenge (not necessary for credit), try to find a solution that only uses one regular expression.

Solution:

```
%%  
== { return T_Equal; }  
=== { return T_StrictEqual; }
```

Match String “====”.